People`s Democratic Republic of Algeria
The Ministry of Higher Education and Scientific Research

University Of Science And Technology Houari Boumediene
Faculty of Computer Science

# Software Testing and Quality Assurance

## Software Testing Methods

Fourth year in Computer Science engineering

By: PhD. HAOUARI Ahmed Taha
Info.ahmed.taha@gmail.com

2025/2026

# Software Testing Approaches

A **testing type** or **approach** defines the high-level strategy for how you examine a software product. It determines whether you are focused on identifying flaws in project documents or validating the actual behavior of the application.

**1.Static testing** is an approach that occurs **without executing the software code**. It focuses on finding errors in documents, logic, and requirements before the application is even built.

**Techniques**: It involves **reviews**, **walkthroughs**, and **inspections** of project artifacts.

**Goal**: To identify defects early in the cycle, such as missing requirements or logical flaws in design, which is highly cost-effective.

Static testing is fundamentally defined as **Verification** because its primary focus is ensuring that the product is being built correctly according to the defined specifications. By examining project artifacts such as requirements, design documents, and source code

# Software Testing Approaches

**Dynamic testing** involves the **actual execution of the software** to find defects. This is the "hands-on" part of the process where you interact with the running application.

**Techniques**: You provide specific inputs and verify if the resulting outputs match the **Expected Results**. This includes techniques like **Black-Box** and **White-Box** testing.

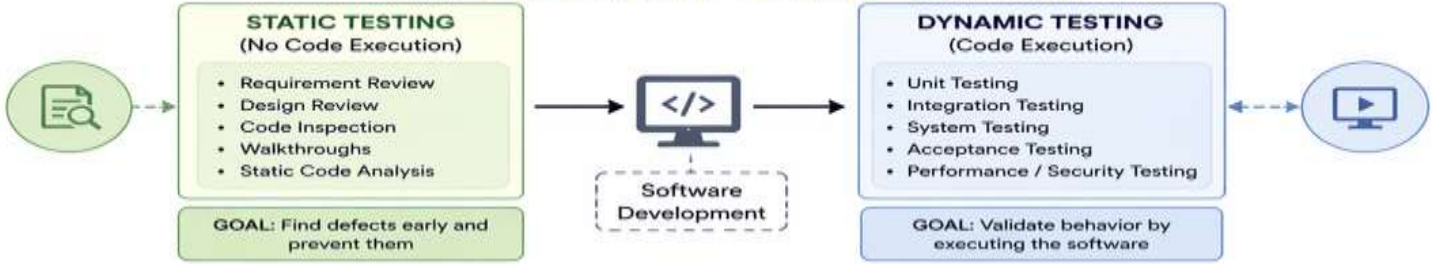
**Goal**: To validate the software's actual behavior, functionality, and performance in a live or simulated environment.
Dynamic testing is fundamentally categorized as **Validation** because it involves the actual execution of the software to ensure that the final product meets the user's needs and business requirements.

Both types are essential to a complete **Software Testing Life Cycle (STLC)**. While static testing helps **prevent errors** by catching them in the requirement phase, dynamic testing provides the final **confidence** that the product works correctly for the end user.

# STATIC TESTING vs DYNAMIC TESTING – COMPARISON

| Aspect | Static Testing | Dynamic Testing |
|---|---|---|
| Definition | Testing performed without executing the software. | Testing performed by executing the software. |
| Execution | No code execution. | Code execution is required. |
| Objective | To find defects early and prevent them before execution. | To find defects by executing the software and validate behavior. |
| Focus | Focus on documentation, design, and code structure. | Focus on the running system and its functionality. |
| When Performed | Early in SDLC (requirements, design, coding phase). | After build is available (integration, system, acceptance phase). |
| Techniques / Examples | • Reviews<br>• Walkthroughs<br>• Inspections<br>• Static code analysis<br>• Requirements review<br>• Design review | • Unit Testing<br>• Integration Testing<br>• System Testing<br>• Acceptance Testing<br>• Regression Testing<br>• Performance Testing |
| Tools | Review checklist, Inspection tools, Static analysis tools (e.g., SonarQube) | Test management tools, Test automation tools (e.g., Selenium, JUnit) |
| Defects Found | Requirements defects, design flaws, code standard violations, logic errors | Functional defects, interface issues, runtime errors, performance issues |
| Cost of Fix | Low (defects found early) | High (defects found late) |
| Main Benefit | Prevents defects, saves time and cost | Validates actual behavior, ensures quality from user perspective |

## STATIC vs DYNAMIC TESTING – DIAGRAM




**KEY TAKEAWAY**

Static Testing = Verification (Are we building the product right?)

Dynamic Testing = Validation (Are we building the right product?)

# Example of Static testing

Let’s try using our previous example

**Requirement Review**

**Requirement:**

“The system shall lock the user after 3 failed login attempts.”

**Static test finding:**

❌ Missing detail: “How long is the lock duration?”

❌ Missing behavior: “Is reset possible via email?”

**Design Review**

**Design says:**

Login screen → authentication module → database

**Static issues found:**

❌ No mention of encryption (password stored as plain text risk)

❌ No handling for SQL injection protection

**Code Inspection (example)**

if(username == dbUser && password == dbPass)

**Static defect:**

❌ Using == instead of .equals() (in Java)

❌ Password comparison in plain text

| Type | Defect Found |
|---|---|
| Requirements | Missing lock policy details |
| Design | No security encryption |
| Code | Wrong string comparison |

# Black-Box Testing

Black-Box Testing (also formally known as **Specification-Based Testing**) is a dynamic testing approach where the test cases are derived solely from the software's external requirements and functional specifications.

Black-box testing focuses only on **inputs and outputs**, which mean, evaluating the system **without any knowledge of its internal implementation, code structure, or logic**.

To achieve high test coverage without executing every possible input, we use the following formal techniques:
**Equivalence Partitioning (EP)**
**Boundary Value Analysis (BVA)**
**Decision Table Testing**

# Black-Box Testing

To have a clear understanding of these methods, let us consider the factorial function as an example.

**Equivalence Partitioning (EP)**

It is a black-box testing technique that **splits the entire input domain** into groups of values called **partitions.** The idea is that the system is expected to handle all inputs within the same **partition in a similar way**. Therefore, **testing a single representative** value from each partition is **considered sufficient**, as it reflects the behavior of the whole group.

Using this logic, we categorize the infinite number of possible inputs into three distinct partition

| Partition Type | Range (Logical Class) | (Test Case) | Expected Outcome |
|---|---|---|---|
| **Invalid Partition** | n < 0 (Negative) | -5 | Error Message(invalide input) |
| **Valid Partition** | >=0  <=12 (Supported Range) | 5 | 120 (Successful) |
| **Invalid Partition** | n > 12 (Out of Range/Overflow) | 15 | Error Message (too large) |

# Black-Box Testing

**Boundary Value Analysis(BVA)** A technique used to identify defects at **the edges of input partitions**. It focuses on testing the minimum, maximum, and values just inside/outside these boundaries, as software is most likely to fail at these transition points.

In programming, most defects occur **at the boundaries of a range** because of "off-by-one" errors (for example, using < instead of <=). BVA is designed specifically to catch these logical mistakes.

If our requirement is 0<=N <= 12, we have two boundaries: **0** (the minimum) and **12** (the maximum). To perform BVA, we test the boundary itself, one step below it, and one step above it.

| Boundary Point | Value | Category | Expected Outcome | Boundary Point | Value | Category | Expected |
|---|---|---|---|---|---|---|---|
| **Below Min** | -1 | Invalid | Error Message | **Below Max** | 11 | Valid | 39,916,800 |
| **Minimum** | 0 | Valid | 1 (Factorial of 0 is 1) | **Maximum** | 12 | Valid (Edge) | 479,001,600 |
| **Above Min** | 1 | Valid | 1 | **Above Max** | 13 | Invalid | Error Message / Overflow |

# Black-Box Testing

**3- Decision Table Testing**

designed to validate complex business logic by considering different combinations of **input conditions** and their **corresponding outputs (actions)**. In an academic context, it is used to represent and analyze the logical **relationships between conditions**. It is especially useful for requirements expressed **in "if–then" form**, as it ensures that all possible **combinations of conditions** are systematically covered and tested.

| Rules | R1 | R2 | R3 | R4 |
|---|---|---|---|---|
| C1 | Y | Y | N | N |
| C2 | Y | N | Y | N |
| A1 | X | | | |
| A2 | | X | X | X |

Assume a simple login system with the following conditions:

**Conditions:**

C1: Username is correct?

C2: Password is correct?

**Actions:**

A1: Login successful

A2: Show error message

**R1 (Y, Y):** Username and password are correct → Login

**R2 (Y, N):** Username correct, password wrong → Error

**R3 (N, Y):** Username wrong, password correct → Error

**R4 (N, N):** Both incorrect → Error

Instead of testing random cases, Decision Table Testing ensures: All possible combinations of conditions are tested systematically.

# Exercise 1

The function power(base, exp) computes the result of raising a number (base) to an exponent (exp).

**Requirement:**

The exponent exp must be a **non-negative integer**.

If exp = 0, the function returns **1**.

If exp = 1, the function returns the **base**.

If exp < 0, the function returns an **error value** (e.g., -1).

Give us the results of your Static Test

Write the test case to handle **Equivalence Partitioning and Boundary Value Analysis(BVA)**

```
int power(int base, int exp) {
int result = 1;

if (exp < 0)
return -1; // invalid input

for (int i = 0; i <= exp; i++)
result = result * base;

return result;
}
```

# Exercise 1

| ID | Type | Location | Description of Issue | Severity | Recommendation |
|---|---|---|---|---|---|
| D1 | Logic defect (Code) | Loop condition | for (i = 0; i <= exp; i++) causes one extra iteration | High | Replace with i < exp |
| D2 | Missing requirement detail | Specification level | No clear definition of input type (int only or real numbers) | Medium | Specify input data type explicitly |
| D3 | Missing requirement detail | Specification level | No defined behavior for base = 0 cases | Medium | Define rules for $0^0$ and $0^n$ |

# Exercise 1

**Equivalence Classes**

EP1: exp < 0 (Invalid)
EP2: exp = 0 (Zero case)
EP3: exp = 1 (Unit case)
EP4: exp > 1 (General valid case)

| TC ID | Base | Exp | Expected Result | Type |
|---|---|---|---|---|
| EP1 | 2 | -3 | -1 (error) | Invalid |
| EP2 | 5 | 0 | 1 | Valid |
| EP3 | 4 | 1 | 4 | Valid |
| EP4 | 2 | 3 | 8 | Valid |

# Exercise 1

**Boundary Values**
Lower invalid boundary: **-1**
Lower valid boundary: **0**
Just above lower boundary: **1**
Small valid value: **2**

| TC ID | Input Base | Input Exp | Boundary Type | Expected Result |
|---|---|---|---|---|
| BVA1 | 2 | -1 | Just below lower boundary | -1 (error) |
| BVA2 | 2 | 0 | Lower boundary | 1 |
| BVA3 | 2 | 1 | Just above lower boundary | 2 |

**1. Conditions**

We define the logical conditions affecting the function:

C1: exp < 0

C2: exp = 0

C3: exp = 1

C4: exp > 1

(Only one of these conditions can be true at a time.)

**2. Actions**

A1: Return 1

A2: Compute power normally

A3: Return error (-1)

| Conditions / Rules | R1 | R2 | R3 | R4 |
|---|---|---|---|---|
| C1: exp < 0 | Y | N | N | N |
| C2: exp = 0 | N | Y | N | N |
| C3: exp = 1 | N | N | Y | N |
| C4: exp > 1 | N | N | N | Y |
| A1 | | X | X | |
| A2 | | | | X |
| A3 | X | | | |

# Exercise 1

| ID | Test Case | Technique | Input (base, exp) | Expected Result | Actual Result | Defect Description | Severity |
|---|---|---|---|---|---|---|---|
| D1 | EP4 | EP | (2, 3) | 8 | 16 | Loop runs one extra iteration (i <= exp) | High |
| D3 | BVA4 | BVA | (2, 2) | 4 | 8 | Boundary failure caused by off-by-one loop error | High |

# Black-Box Testing

**6. Final Conclusion**

The main defect is a **loop boundary error (i <= exp)**, which affects all cases where exp > 1.
EP and BVA both successfully detected functional and boundary failures.
Decision Table ensured full logical coverage of exponent behavior.
The system is **not fully correct due to a critical logic defect**, but the test process achieved full coverage of all testing levels.

**7. Final Result**

✓ Test design: Complete
✕ Implementation: Contains critical defect
✓ Testing: Successfully identified all major issues

# White-Box Testing

White-Box Testing (also known as **Structural, Glass-Box, or Clear-Box Testing**) focuses on **"How the software does it."**

it is a method where the tester has **full visibility into the internal logic**, control flow, and data structures of the **source code**.

The primary objective is to verify **the internal health of the code** by ensuring that all internal **paths, loops, and conditions** are executed correctly.

The main goal of white-box testing is to ensure that the internal logic of the software is correct and fully exercised. This includes:

- **Verifying all possible execution paths**
- **Example :**

Test case 1: x = 5 → executes **if branch**

Test case 2: x = -3 → executes **else branch**

```
if (x > 0) {
System.out.println("Positive");
} else {
System.out.println("Non-positive");
}
```

# White-Box Testing

- **Checking conditions, loops, and branches logics**

Example:
Test case 1: n = 3 → loop runs (1, 2, 3)
Test case 2: n = 0 → loop does **not execute**
**Here we Test both loop execution and skipping.**

```
for (int i = 1; i <= n; i++) {
    System.out.println(i);
}
```

- **Ensuring proper data flow and variable usage**

Example:
Test case: x = 5 → output = 5
which Ensures:
Variable initialization
Correct update and usage
No undefined variables

```
int sum = 0;
sum = sum + x;
System.out.println(sum);
```

# White-Box Testing

- **Detecting hidden errors in the code structure**

Test case: x = 12 → only "Big" is printed
The last condition is **never executed** → logical error detected.

```
if (x > 10) {
    System.out.println("Big");
} else if (x > 5) {
    System.out.println("Medium");
} else if (x > 10) { // ✕ unreachable condition
    System.out.println("Again Big");
}
```

# Core White-Box Techniques

**1. Statement Coverage**
This technique ensures that **every single line of code** is executed at least once.
**Goal:** Find lines that are "dead code" or unreachable.

```
public static int factorial(int n) {
if (n < 0) {
return -1; // error case
} else if (n == 0) {
return 1; // 0! = 1
== 1) {
1! = 1

= 1;
2; i <= n; i++) {
sult * i;

ult;
```

In our factori
return -1; →
return 1; → v
return 1; → v
result = 1;
loop body: re
return result;
To achieve 1
**least once**.

| Test Case | Input n | Covered Statements |
|---|---|---|
| TC1 | -1 | Executes return -1 |
| TC2 | 0 | Executes return 1 (n=0 branch) |
| TC3 | 1 | Executes return 1 (n=1 branch) |
| TC4 | 3 | Executes initialization + loop + return result |

# Core White-Box Techniques

**2. Branch/Decision Coverage**

This focuses on the "decision points**" (the if statements).** It ensures that every branch (both the **True** and the **False** path) is tested.

From our example, we have this branch to test

•(n < 0) → TRUE / FALSE

•(n == 0) → TRUE / FALSE

• (n == 1) → TRUE / FALSE

• (i <= n) (loop condition) → TRUE / FALSE

| Test Case | Input n | Covered Branches |
|---|---|---|
| TC1 | -1 | (n < 0) = TRUE |
| TC2 | 0 | (n < 0) = FALSE, (n == 0) = TRUE |
| TC3 | 1 | (n < 0) = FALSE, (n == 0) = FALSE, (n == 1) = TRUE |
| TC4 | 3 | All previous = FALSE + loop TRUE and FALSE |

# Core White-Box Techniques

**2. Branch/Decision Coverage**

This focuses on the "decision points**" (the if statements).** It ensures that every branch (both the **True** and the **False** path) is tested.

From our example, we have this branch to test

•(n < 0) → TRUE / FALSE

•(n == 0) → TRUE / FALSE

• (n == 1) → TRUE / FALSE

• (i <= n) (loop condition) → TRUE / FALSE

| Test Case | Input n | Covered Branches |
|---|---|---|
| TC1 | -1 | (n < 0) = TRUE |
| TC2 | 0 | (n < 0) = FALSE, (n == 0) = TRUE |
| TC3 | 1 | (n < 0) = FALSE, (n == 0) = FALSE, (n == 1) = TRUE |
| TC4 | 3 | All previous = FALSE + loop TRUE and FALSE |

# Core White-Box Techniques

**3. Path Testing**

This is the most comprehensive technique. It involves identifying and executing **every possible unique path (feasible paths)**through the code from start to finish.

**Independent Paths for factoriel**

**Path 1: Negative input** $n < 0 \rightarrow$ return -1

**Path 2: Zero case** $n \geq 0$ $n == 0 \rightarrow$ return 1

**Path 3: One case** $n \geq 0$ $n \neq 0$ $n == 1 \rightarrow$ return 1

**Path 4: Loop NOT executed** $n \geq 2$ but loop condition false immediately **Impossible here** (because loop starts at i = 2, so for $n \geq 2$, it executes at least once)

**Path 5: Loop executed** $n \geq 2$ loop runs at least once → return result

| Test Case | Input n | Covered Path |
|---|---|---|
| TC1 | -1 | P1 |
| TC2 | 0 | P2 |
| TC3 | 1 | P3 |
| TC4 | 3 | P5 |

# Core White-Box Techniques

In programs with loops, **full path coverage is often impossible** (because loops can execute infinite times) So we usually test:

•0 iterations (if possible)
• at least 1 iteration
• or multiple iterations

**Note In our example, the three techniques gave us almost the same test cases, However it's not often the case**

| Method | What it guarantees |
|---|---|
| **Statement Coverage** | Every instruction is executed |
| **Branch Coverage** | Every decision (TRUE/FALSE) is tested |
| **Path Coverage** | Every full path (combination of decisions) is tested |

# Core White-Box Techniques

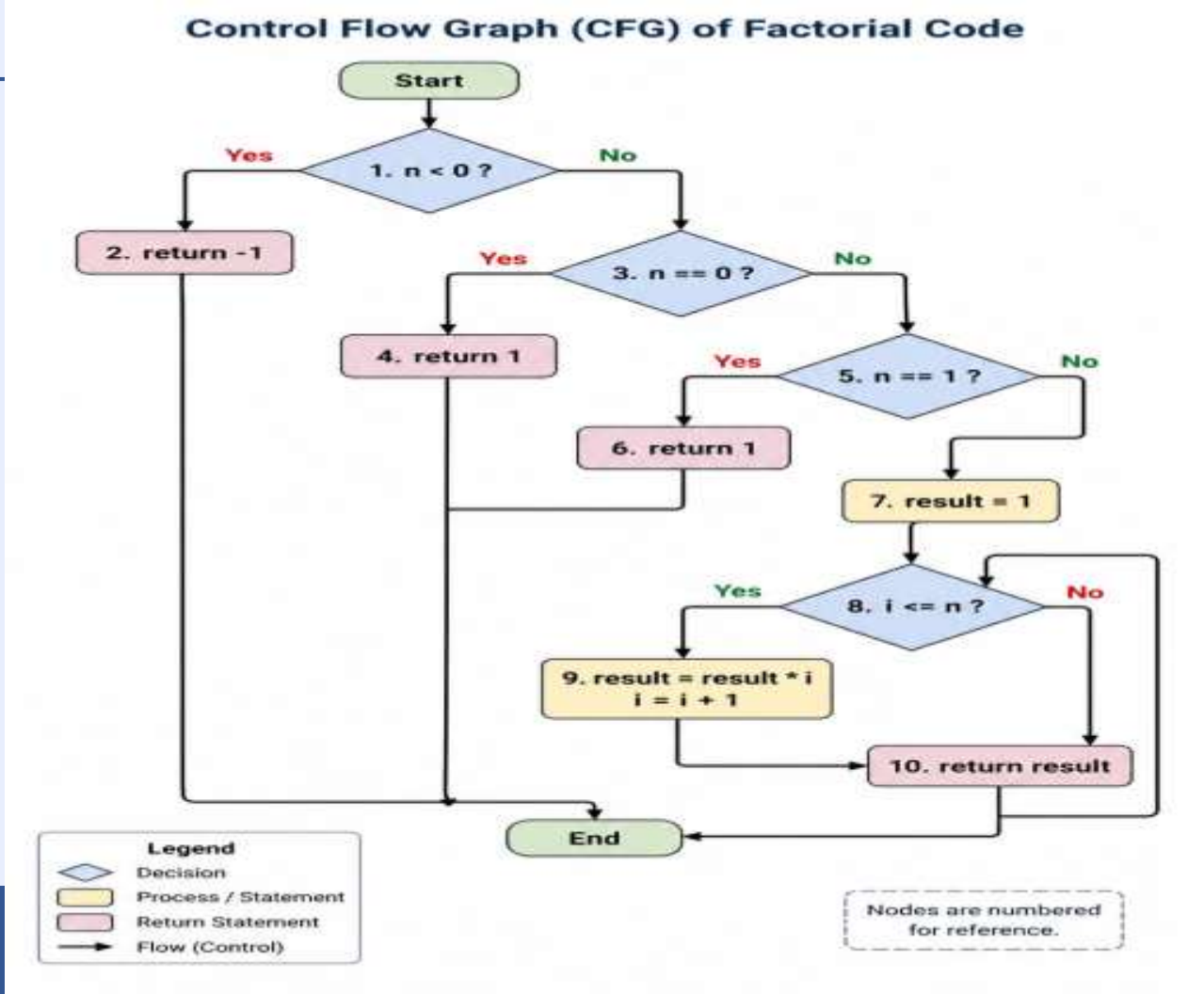
Let’s try another example :
x > 0 → TRUE / FALSE
y > 0 → TRUE / FALSE
Test cases:
**(x=1, y=1)**
**(x=-1, y=1)**
**(x=1, y=-1)**
**The paths**
P1 x ≤ 0
P2 x > 0 AND y ≤ 0
P3 x > 0 AND y > 0

```
if (x > 0) {
        if (y > 0) {
                System.out.println("A");
                }
        }
```

# Control Flow Graph (CFG)

a CFG is a graphical representation of all paths that might be traversed through a program during its execution.
In a CFG:
**Nodes:** Represent basic blocks of executable code (statements).
**Edges:** Represent the transfer of control (the flow) between these blocks.
**Decision Nodes:** Points where the graph splits (like if statements or loops).

The most important reason we use a CFG is to calculate **Cyclomatic Complexity (V(G))**.


Control Flow Graph (CFG) of Factorial Code
Start
1. n < 0 ?
Yes
No
2. return -1
3. n == 0 ?
Yes
No
4. return 1
5. n == 1 ?
Yes
No
6. return 1
7. result = 1
8. i <= n ?
Yes
No
9. result = result * i
i = i + 1
10. return result
End
Legend
Decision
Process / Statement
Return Statement
Flow (Control)
Nodes are numbered for reference.

# Cyclomatic Complexity

**Cyclomatic Complexity (CC)** is a **software metric** used to measure **the complexity of a program**. It counts the number of logical "**independent paths**" through your source code.
Think of it as a score:
**A low CC :** means the code is simple, easy to test, and easy to maintain.
**A high CC** means the code has many decisions (if-statements, loops, switch-cases), making it complex and more likely to contain bugs.
To find the complexity of a **Control Flow Graph (CFG)**, we use this standard formula:

$$CC = E - N + 2*P$$

**E** = The number of **Edges** (the lines/arrows connecting the code).
**N** = The number of **Nodes** (the code statements or points).
**P** = Connected components (for a single method, this is always **1**).

Or

$$V(G)=D+1$$

**D=** number of **decision nodes**

# Cyclomatic Complexity

**In our example:** N=12 and E=15 and P=1 or we use the simple version and we have D=4
So we have **CC= E-N+2*P=15-12+2 = 5**

**or CC=D+1=4+1= 5**

**What this means for a tester:**
A complexity of **5** tells you that you need exactly **5 test cases** to cover every unique path in this method or a function . If you only write 4 tests, you are missing an entire branch of logic.
**Why does this matter?**
The industry standard suggests that a single method should have a complexity of **10 or less**. If a method has a complexity of 25, it is too "busy" and should be broken down into smaller, simpler methods to reduce the risk of errors.

# Exercise 2

From the code shown at the right:

**1. Draw the Control Flow Graph (CFG) of this program.**
**2. Identify all independent execution paths.**
**3. Determine the cyclomatic complexity of the program.**
**4. Provide the minimum set of test cases to achieve:**

100% **Statement Coverage**
100% **Branch Coverage**
100% **Path Coverage**

```
public static void process(int x, int y) {

    if (x > 0) {
        if (y > 0) {
            System.out.println("A");
        } else {
            System.out.println("B");
        }
    } else {
        System.out.println("C");
    }

    System.out.println("End");
}
```

# Exercise 2

**2. Independent Paths**

**Path 1:** $x \le 0 \rightarrow C \rightarrow End$

**Path 2:** $x > 0 \;\; y > 0 \rightarrow A \rightarrow End$

**Path 3:** $x > 0 \;\; y \le 0 \rightarrow B \rightarrow End$

**Total Paths = 3**

**3. Cyclomatic Complexity**

V(G)= CC=D+1=**2+1**

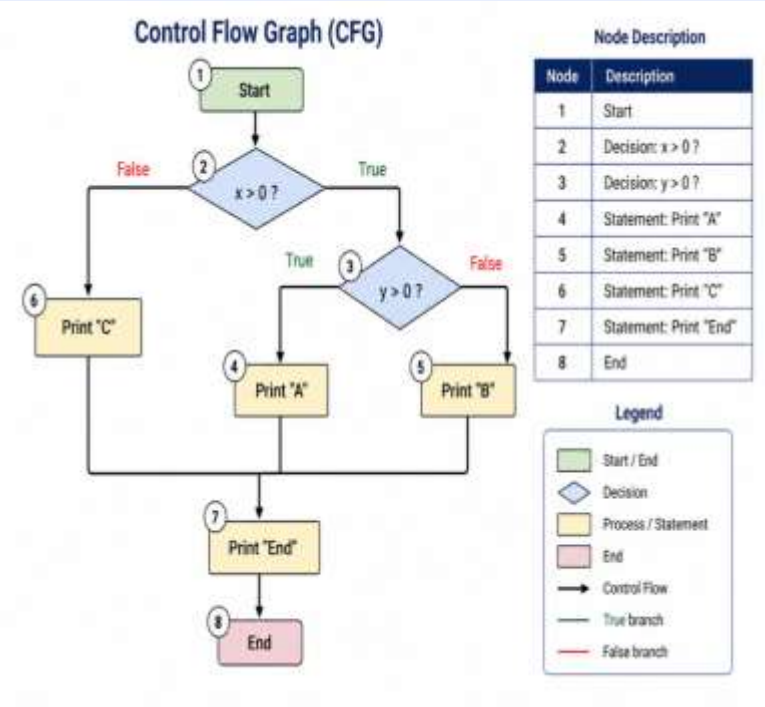
**Result:**

**Cyclomatic Complexity = 3**

**4--A. Statement Coverage (100%)**

Goal: execute all statements (A, B, C, End)

| TC | x | y | Output |
|---|---|---|---|
| TC1 | 1 | 1 | A and end |
| TC2 | 1 | -1 | B and end |
| TC3 | -1 | 0 | C and end |



# Exercise 2

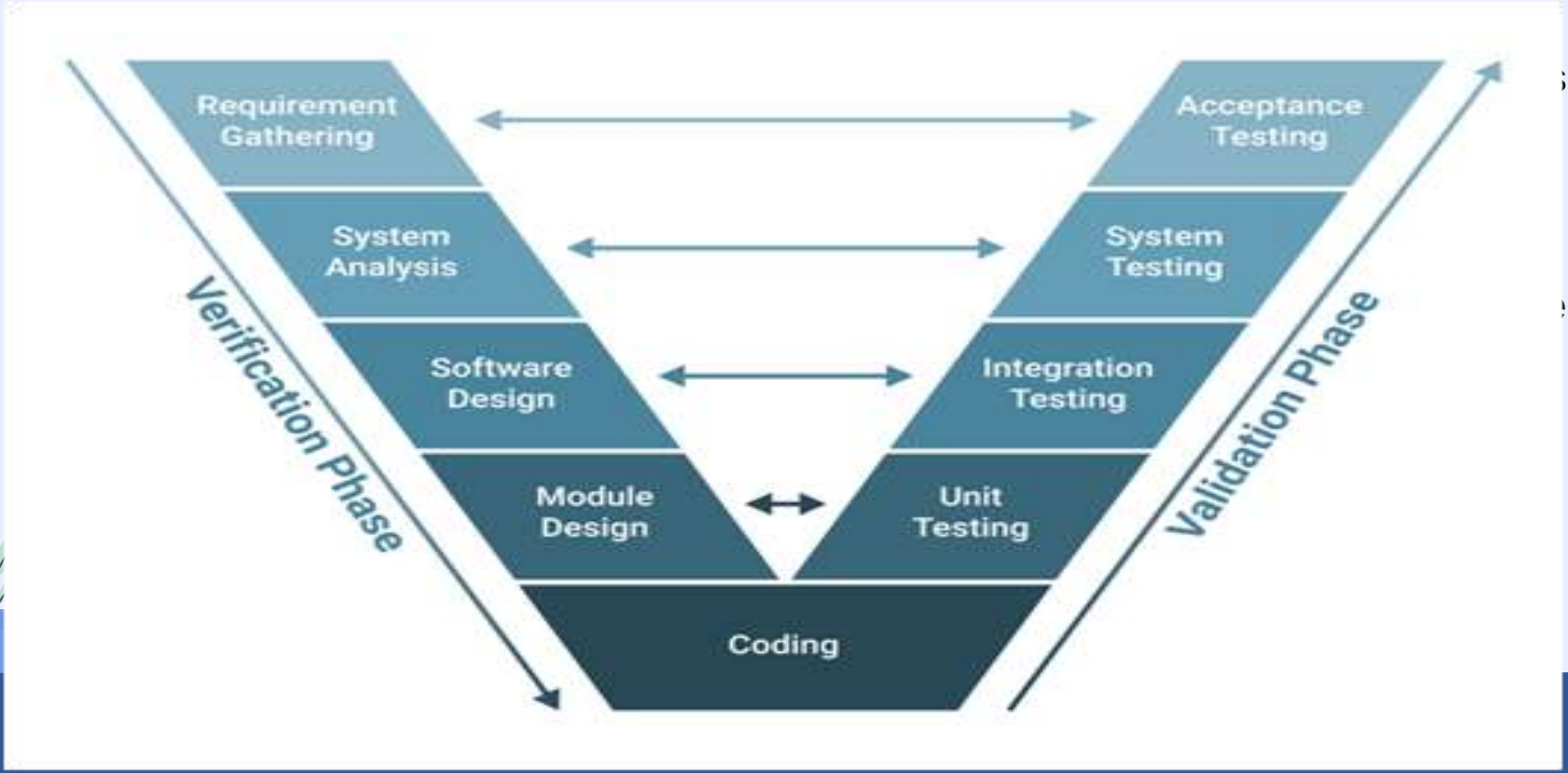
**B. Branch Coverage (100%)**
Goal: each decision TRUE and FALSE, in our case, all the decision are covered by the previous tests cases

| Decision | Already Covered by |
|---|---|
| x > 0 TRUE | TC1, TC2 |
| x > 0 FALSE | TC3 |
| y > 0 TRUE | TC1 |
| y > 0 FALSE | TC2 |

**C. Path Coverage (100%)**
Goal: cover all paths

| Path | Test Case (X,Y) |
|---|---|
| P1: C | (-1, 0) |
| P2: A | (1, 1) |
| P3: B | (1, -1) |

# Level of Testing



# Level of Testing

**1. Unit Testing**
This is the first level of testing. It focuses on the smallest "units" of code in isolation, such as a single Java method or class.
**Goal:** To verify that each individual component works correctly.
**Who performs it:** Developers.
**Example (Java):** Testing specifically that your factorial(5) method returns exactly 120.

**2. Integration Testing**
After individual units are tested, they are combined. Integration testing focuses on the **interfaces** and the **flow of data** between these units.
**Goal:** To expose defects in the interaction between integrated components.
**Who performs it:** Developers or specialized testers.
**Example (Java):** Testing that the FactorialUI class correctly passes a number to the FactorialCalculator class and displays the result on the screen.

# Level of Testing

**3. System Testing**
This level tests the **complete and integrated software product** as a whole. **It is a "Black-Box"** level because we are testing the system's compliance with the original requirements.

**Goal:** To validate the end-to-end system specifications, including functionality, security, and performance.

**Who performs it:** Independent Testing Team (QA).
**Example:** Installing the Factorial App on a phone and checking if it opens, calculates correctly, handles "low battery" interruptions, and saves user history.

# Level of Testing

**4. Acceptance Testing**
This is the final level before the software goes "Live." It focuses on the business needs rather than just technical requirements.

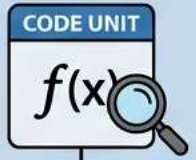
**Goal:** To establish **confidence** in the system and ensure it is ready for deployment.

**Two Main Types:**
**Alpha Testing:** Done by the company's internal employees in a test environment.
**Beta Testing:** Done by actual customers or end-users in their own environment.
**Who performs it:** Customers, end-users, or product owners.
**Example:** A math teacher uses the Factorial App and confirms that the user interface is easy enough for students to use in a classroom.

# THE FOUR LEVELS OF SOFTWARE TESTING

## STAGES OF ORGANIZED AND EXECUTED TESTING

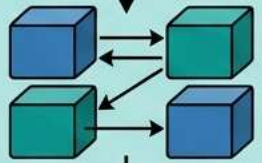
**UNIT LEVEL**




**1. UNIT TESTING (DEVELOPER) -**
Validates individual components in isolation isolation (e.g., specific Java methods).

UNITS COMBINED →

**INTEGRATION LEVEL**



**2. INTEGRATION TESTING (TESTERS/DEVS) -**
Exposes defects in the flow of data between combined units.

UNITS COMBINED → INTEGRATED APPLICATION →

**SYSTEM LEVEL**

**3. SYSTEM TESTING (QA TEAM) -**
Tests the integrated software as a whole for functionality, security, performance.

COMPLETED APPLICATION →

**ACCEPTANCE LEVEL**

**4. ACCEPTANCE TESTING (CLIENTS/USERS) -**
Final check before "Live" deployment to ensure it meets user and business needs.

WHITE-BOX FOCUS

BLACK-BOX FOCUS

FULL CONFIDENCE & DEPLOYMENT →

# Level of Testing

•**Non-Functional Testing** is defined as the type of testing that verifies the "attributes" **or "qualities" of the software**, rather than specific behaviors or functions.

•While functional testing asks, *"Does the Java method calculate the correct factorial?"*, non-functional testing asks, *"How fast does it calculate it?"* or *"Can 500 users calculate factorials simultaneously without the server crashing?"*

•These tests are critical because a program can be functionally perfect but still fail in the market if it is too slow, easily hacked, or confusing to use.

•Here the testers ensure that the **software provides** a high-quality user experience under real-world conditions.

# Level of Testing

**Performance Testing:** Checks the speed, responsiveness, and stability of the system.
*Example:* Using a tool like JMeter to simulate 1,000 users calling your FactorialCalculator Java API at once to measure the response time.

**Security Testing:** Identifies vulnerabilities, threats, and risks in the software to prevent malicious attacks.
*Example:* Testing if an attacker can perform an "Injection" attack through the input field of your Java application to bypass login screens.
**Usability Testing:** Evaluates how easy and intuitive the user interface (UI) is for the end-user.
*Example:* Observing a user as they navigate your Factorial app to see if they can find the "Reset" button without looking at a manual.

# Level of Testing

**Reliability Testing:** Verifies if the software can perform a failure-free operation for a specified period in a particular environment.
*Example:* Running the Java application continuously for 72 hours to ensure it doesn't crash or suffer from a "Memory Leak" where the RAM usage slowly climbs until the system dies.

**Compatibility Testing:** Ensures the software works across different hardware, operating systems, and browsers.
*Example:* Verifying that your Java-based web app looks and functions exactly the same on Chrome, Firefox, and Safari, and on both Windows and macOS.

ing.a.haouari@gmail.com